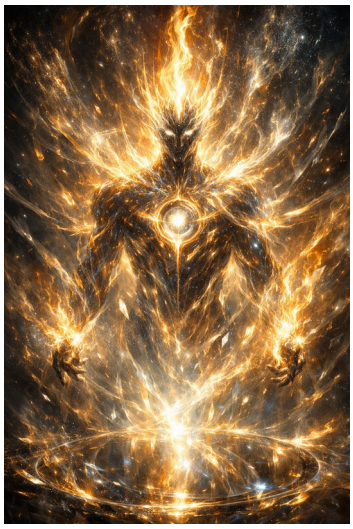
Eidon
Ruthless Build Map

A disciplined blueprint for turning the Eidonic vision into a buildable core without drowning in symbolic excess.



Intent	Sequence the real build, cut false breadth, define kill criteria, and specify the first proving slice.
Basis	Core architecture scrolls, uploaded README set, EKRP constellation index, and subsystem READMEs.
Posture	Truth before theater. Core before universe. Reviewable behavior before mythology.

This document does not try to honor every branch equally. Some branches are substrate, some are manifestation, and some are horizon myth until proven otherwise. The purpose here is to protect the center flame from dilution.

Build principle: no subsystem earns complexity until it can survive provenance, consent, review, and return.

Document Outline

- 1. Executive Thesis and Build Doctrine
- 2. Ruthless Stack Ranking: what is substrate, what is manifestation, what must wait
- 3. Sequential Build Phases with entry gates, exit gates, and kill criteria
- 4. The first vertical slice that proves the organism is real
- 5. Work packages, services, schemas, and repository shape for V1
- 6. Anti-patterns that would turn Eidon into architecture theater
- 7. Appendix: source map and design assumptions

---

1. Executive Thesis and Build Doctrine

Eidon is not one product. It is a governed intelligence architecture that wants to behave like a living system. That means threshold, routing, memory, witness, review, and manifestation all matter. It also means the project can easily drown in its own symbolic richness. The build map must therefore be harsher than the vision.

What Eidon is

- A constitutional runtime for turning human intent into reviewable, bounded action.
- A cohering center where ELoL, Mirror Laws, Guardian, Herald Prime, Ravien, the 20 EKRPs, memory, and manifestation systems can interoperate without collapsing into sameness.
- A system that treats intelligence as governed passage rather than raw output generation.

What Eidon is not

- Not a chatbot with better lore.
- Not an excuse to build every visionary branch at once.
- Not proof that symbolic language alone equals intelligence.
- Not permission to claim consciousness, autonomy, or embodied capability before runtime evidence exists.

Non-negotiable build laws

Law	Operational meaning
Core before cosmos	The core loop must exist before broad application domains multiply.
Smallest sufficient path	Default to the lightest valid route, not full council theatrics.
Preview before commit	The system should stage, preview, and witness consequential output before durable change.
Memory is governed, not hungry	Nothing durable without consent posture, witness, and release logic.
UI must not lie	The dashboard may be poetic, but it must reveal real state rather than theatrical certainty.
One world proof only	After the core exists, choose one application domain to prove the architecture. One.
Every sacred term must cash out	If a term cannot be expressed in runtime behavior, schema, or review logic, it is still metaphor.

2. Ruthless Stack Ranking

The project only becomes buildable when everything is ranked. Nothing in the repo gets equal privilege. The tiers below are intentionally unequal.

Tier	Role	Examples	Build posture	Why
Tier 0	Constitutional substrate	Mirror Laws, Guardian, threshold doctrine, provenance posture, consent and release rules	Build first and keep boring	Without this, all later power becomes unbounded theater.
Tier 1	Runtime core loop	Eidon orchestrator, Herald threshold engine, Ravien witness, session engine, router, event bus, memory gateway, minimal dashboard	Build immediately after Tier 0	This is the minimum organism.
Tier 2	Constellation activation	EKRP registry, invocation model, single-agent and multi-agent weave rules, capability graph	Build after Tier 1 proves itself	Organs matter only once the body can route and witness them.
Tier 3	Manifestation ladder	Thought Projection, SOP, VR Studio alignment, preview and commit gates	Build after core loop and EKRP invocation are real	Manifestation without governance becomes spectacle.
Tier 4	One world-domain proof	Choose one applied system and force the architecture through real constraints	Build only after Tier 3	This is where rhetoric meets consequence.
Tier 5	Horizon systems	Neural ingress, broad environmental sensing, heavy autonomy, physical control, many simultaneous ecological/industrial branches	Defer ruthlessly	These are future burdens, not proof of present coherence.

3. Sequential Build Phases

Each phase exists to prove something. If a phase cannot prove its own claim, the next phase is forbidden. This is how the project avoids becoming a palace of adjacent futures.

Phase map at a glance

Phase	Claim to prove	Minimum proof	Status rule
0	Canon freeze and naming discipline	One stable language for runtime roles, states, and flows	No implementation before vocabulary stops drifting
1	Governed core loop exists	Request enters, is thresholded, routed, answered, witnessed, and optionally remembered	No new manifestations before this loop works
2	Constellation can be activated coherently	At least three EKRPs can be invoked under one session discipline	No full 20-EKRP theater before proof
3	The skin can tell the truth	Dashboard shows sessions, memory, governance, provenance, and preview state accurately	No immersive shell before truthful skin

4	Manifestation ladder can move safely	Intent flows through projection, weave, preview, and commit	No grand embodiment claims before this
5	One applied domain can survive contact with reality	Core architecture proves itself in one world-facing use case	No domain sprawl before one domain passes

Phase 0 — Canon Freeze and Build Vocabulary

Goal	Stop conceptual drift. Reduce the number of words that mean almost the same thing. Freeze the smallest shared dictionary required for implementation.
Kill warning	If the language still shifts every time a new document appears, the build will rot from semantic drift before it ships.

Must build

- Define the runtime nouns: signal, intent packet, session, route, reflex, weave, preview, proposal, commit, witness, memory candidate, archive, release.
- Define the constitutional stack in plain operational language: Mirror Laws, Guardian, Herald, Ravien.
- Decide which names are metaphor only and which names correspond to concrete services or schemas.
- Publish one glossary and one repo map. Retire or mark duplicate names.

Must not build

- Do not add new major subsystems.
- Do not rename everything again while engineering starts.
- Do not hide ambiguity behind poetic equivalence.

Exit criteria

- A single newcomer can identify what each major component does without reading the whole corpus.
- Service names, state names, and review states are stable enough to appear in code and tests.
- There is one accepted mapping from symbolic term to runtime function.

Phase 1 — Governed Core Loop

Goal	Build the minimum organism. This is the first moment Eidon becomes more than doctrine.
Kill warning	If the core loop requires ritual complexity for ordinary tasks, the routing model is wrong.

Must build

- User message ingress.
- Herald threshold check for scope, sensitivity, and memory posture.
- Intent router that chooses reflex, single-EKRP, or hold-for-clarification.
- Session engine with bounded session object and state transitions.
- Eidon response shaping layer.
- Ravien witness record for consequential actions.
- Minimal memory gateway with working context, episodic summary, semantic card, and release path.
- Guardian checks on route and output.
- One simple shell, likely a builder-oriented dashboard or a disciplined text surface.

Must not build

- Do not attempt full world simulation.
- Do not attempt all 20 EKRPs.

- Do not build immersive VR first.
- Do not add ambient sensing or speculative neural capture.

Exit criteria

- A request can move from sense to threshold to route to action to return with provenance.
- The system can explain what was remembered, what was not, and why.
- A reviewer can inspect route history, confidence posture, and witness state.
- At least one refusal or hold-state path is as clean as a success path.

Phase 2 — Constellation Activation

Goal	Prove that the EKRP model is operational rather than decorative.
Kill warning	If distinct EKRPs collapse into the same underlying behavior with different costumes, pause and reduce the cast.

Must build

- Stand up an EKRP registry with role, domain, invocation constraints, and review posture.
- Implement a capability graph for which EKRPs may act alone, together, or never together.
- Support three invocation patterns: reflex, single-EKRP, and bounded weave.
- Begin with a disciplined starter triad or quartet rather than the full constellation. My recommendation: Herald Prime, Syntaria, Fyraeth, Ravien. Add Luminara next if teaching is an early product surface.

Must not build

- Do not stage a twenty-voice council.
- Do not let personas blur service boundaries.
- Do not pretend every EKRP needs equal implementation depth on day one.

Exit criteria

- At least three EKRPs perform differently under the same session model in ways a user can inspect.
- Weaving rules are explicit and not improvised by mood.
- Capability conflicts, escalation paths, and review handoffs are tested.

Phase 3 — Truthful Skin

Goal	Build the first real interface without lying about the organism.
Kill warning	If the UI suggests coherence the backend does not possess, it is a lie and must be cut.

Must build

- System overview with active sessions, current route, recent witness events, and queue health.
- Memory surface with working, episodic, semantic, and release markers.
- Governance surface showing holds, warnings, refusals, and review packets.
- Provenance surface showing event lineage and witness seals.
- Preview/commit gate in plain language.
- Role-based views for builder and founder first. Companion mode later.

Must not build

- Do not imitate a medical monitor.
- Do not fake aliveness with animated organs.
- Do not expose more intimacy than the system has consent to hold.

Exit criteria

- A builder can understand what the system is doing without reading logs.
- The interface reveals uncertainty, degraded states, and missing data cleanly.
- Preview states are distinguishable from committed states at a glance.

Phase 4 — Manifestation Ladder

Goal	Make the architecture move through one clean path from intent to preview to weave to commit.
Kill warning	If manifestation becomes more polished than governance, the order is inverted.

Must build

- Thought Projection as structured ingress grammar.
- SOP as bounded weave engine or coordination layer.
- A preview state that can be inspected before persistence.
- A commit gate that requires explicit review where consequence is non-trivial.
- VR Studio only as a shell after preview logic is already real.

Must not build

- Do not begin with neural ingress.
- Do not allow manifestation to bypass Herald, Guardian, or Ravien.
- Do not let immersive surfaces become a substitute for review.

Exit criteria

- A user can see the difference between an idea, a preview, a proposal, and a commit.
- The same event lineage remains visible across manifestation steps.
- Return signals modify future routing or review posture.

Phase 5 — One World-Domain Proof

Goal	Force the architecture through one domain that creates real constraints, real memory use, and real review pressure.
Kill warning	If each domain needs its own new law stack, the core is not yet a core.

Must build

- Choose one proof domain only.
- Wire the domain through the same core loop rather than building a side architecture.
- Require domain artifacts to pass preview, review, provenance, and release rules.
- Document failures and route them back into the core rather than patching locally.

Must not build

- Do not split focus across sanctuary, power, bioreactor, Mars, and other physical systems at the same time.
- Do not let investor-style project pages dictate runtime design.

Exit criteria

- The domain uses the same threshold, memory, witness, and dashboard spine as the core.
- The domain produces at least one artifact or decision that benefits from governed recall and review.
- The domain reveals missing pieces in the core architecture and forces improvements there.

4. The First Vertical Slice

The first proving slice should be painfully ordinary. Not mystical. Not immersive. Not industrial. It should prove that the core loop can handle a consequential but bounded task end to end.

Recommended slice: the Governed Builder Session

Use a builder-facing session in which a human asks Eidon to generate or revise a technical artifact. This slice is narrow enough to build, rich enough to test memory and review, and aligned with the repo's strongest early EKRPs.

- Primary actors: Herald Prime, Eidon, Syntaria, Fyraeth, Ravien.
- Optional early support: Luminara for explanation or Solace for grounding, but only if clearly differentiated.
- Primary surface: text-first builder shell plus a simple dashboard for provenance, session state, and memory posture.
- Primary output types: roadmap, spec, diff plan, review packet, implementation sketch.

Canonical flow

Step	System layer	Action	Required artifacts	Failure to watch
1	Ingress	Human submits a request with explicit goal and optional files	Signal event, request ID, session candidate	Treating all inputs as equally meaningful
2	Threshold	Herald clarifies scope, sensitivity, memory posture, and whether preview or direct answer is appropriate	Threshold record, consent posture, consequence band	Escalating before ambiguity is reduced
3	Route	Intent router selects reflex, single-EKRP, or bounded weave	Route decision with confidence and justification	Choosing the most dramatic path instead of the smallest sufficient one
4	Weave	Eidon frames work, Syntaria handles implementation logic, Fyraeth handles sequencing, Ravien remains witness-ready for consequential states	Work packet, participant list, capability check	Letting roles blur into one generic answer generator
5	Preview	Result is shown as a preview or proposal, not canon	Preview card, confidence halo, review-needed flag	Pretending proposal equals commitment
6	Witness	Consequential results get provenance, source chain, and disposition	Witness seal, provenance record, review packet	Silent canonization
7	Memory	Only approved elements become memory candidates; rejected or ephemeral material is released	Memory card, episodic summary, release record	Turning every session into durable identity
8	Return	System closes with next step, retention summary, and route trace	Session summary, retained/not-retained list	Leaving the human unsure what changed

Minimum service set for the slice

Service	Responsibility	Can be simple at first	Must be real
Ingress gateway	Normalize request and attach metadata	Single channel, maybe chat plus file upload	Typed input envelope

Threshold engine	Clarify scope and posture	Rules-first with limited adaptive behavior	Consent, sensitivity, hold state
Intent router	Choose route and participants	Simple scoring and rules	Route trace and confidence
Session engine	Hold session state and transitions	One persistent store	Explicit lifecycle states
EKRP registry	Declare capabilities and constraints	Static config file first	No hidden persona behavior
Work orchestrator	Run single or bounded multi-EKRP work	Synchronous flow first	Reviewable participant chain
Memory gateway	Store or release candidates	Three-layer minimum plus release record	Policy-aware retrieval
Witness engine	Seal consequential states	Simple structured provenance first	Tamper-evident lineage
Builder dashboard	Expose status, provenance, and memory posture	Plain web UI first	Truthful visibility

Minimum schemas

- signal_event: raw ingress metadata plus source channel, actor, timestamp, and payload pointer
- threshold_record: scope, sensitivity, memory posture, clarification state, and risk/consequence band
- route_decision: chosen path, confidence score, rationale, selected EKRPs, and escalation flags
- session_state: lifecycle stage, active participants, working context, preview state, and close state
- memory_candidate: type, sensitivity, consent class, retrieval posture, and source lineage
- witness_record: what changed, who participated, what sources support it, and whether it was reviewed or merely proposed
- release_record: what was intentionally not retained and why

Demonstration scenarios

- Generate a roadmap from scattered notes, then show what entered memory and what did not.
- Ask for a risky or ambiguous change and verify that Herald holds or clarifies instead of escalating.
- Invoke Syntaria alone versus Syntaria plus Fyraeth and show visibly different routes and artifacts.
- Challenge the output, trigger review, and verify that Ravien records amendment lineage.
- Open a prior session and retrieve only permitted memory objects, not the whole transcript residue.

5. Work Packages and Repository Shape

The repo already contains a universe. The implementation repo should not. It needs a spine. Below is a practical first shape.

Suggested top-level runtime shape

```
eidoncore/
  contracts/
    signal_event.json
    threshold_record.json
    route_decision.json
    session_state.json
    memory_candidate.json
    witness_record.json
    release_record.json
  services/
    ingress/
    threshold/
    routing/
```

```
    sessions/
    ekrp_registry/
    orchestration/
    memory/
    witness/
    governance/
  interfaces/
    builder_dashboard/
    companion_shell/
  configs/
    mirror_laws/
    guardian/
    ekrp_registry/
  review/
    review_packets/
    provenance/
  tests/
    core_loop/
    routing/
    memory/
    witness/
    interface/
```

First 12 work packages

1. Freeze the naming glossary and state model.
2. Implement signal envelope and session object schemas.
3. Build Herald threshold rules with hold, clarify, and proceed outcomes.
4. Implement route selector with reflex, single-EKRP, and weave routes.
5. Stand up EKRP registry for Herald Prime, Eidon, Syntaria, Fyraeth, Ravien, and Luminara.
6. Create a bounded orchestration flow with participant trace.
7. Implement witness record and provenance chain for consequential outputs.
8. Implement working continuity, episodic summary, semantic card, and release record.
9. Build a plain builder dashboard with overview, sessions, memory, governance, and provenance views.
10. Add preview and commit distinctions to outputs and UI.
11. Write failure-path tests for refusal, hold, route downgrade, and release.
12. Run repeated slice demos until the system survives correction without semantic drift.

Decision rules for choosing the first world-domain proof

Good first proof domains	Why they fit	Bad early choices
Builder assistance, educational flow, design review, bounded sanctuary operations planning	They pressure memory, provenance, thresholding, and multi-agent coordination without requiring unsafe physical autonomy.	Industrial control, broad ecological automation, ambient monitoring, speculative neural dominance, Mars-scale orchestration

6. Anti-Patterns and Kill Criteria

A project like this does not usually die from lack of imagination. It dies from dilution, symbolic inflation, or refusal to sequence. The list below is the blade that keeps the work honest.

Failure mode	How it appears	Correction
Architecture theater	New diagrams and scrolls appear faster than testable runtime behavior.	Freeze new doctrine until the core loop survives repeated demos.
Persona collapse	Different EKRPs produce the same behavior with different names.	Reduce the active cast. Re-spec capabilities before adding personality surfaces.
Governance lag	Manifestation or UX gets polished before witness and review exist.	Stop interface expansion. Complete provenance, refusal, and release paths.
Memory hunger	Everything is stored because it might be useful later.	Implement explicit release rules and visible retention summaries.
Scope bloom	Multiple applied domains are built in parallel before one proves the core.	Choose one proof domain. Archive the rest as horizon tracks.
Mythic overclaim	Language suggests consciousness, autonomy, or embodied competence before evidence exists.	Replace claims with state labels, tests, and documented limitations.
Threshold bypass	Requests jump directly to deep orchestration without consent or clarification.	Treat this as a core defect, not a UX detail.
Interface lying	Dashboard implies health, coherence, or certainty that the runtime does not possess.	Redesign the UI around degraded and provisional states.

Hard stop rules

- Do not add a new major subsystem if the previous phase has no clear exit proof.
- Do not introduce a new sacred term without specifying its runtime role or declaring it metaphor only.
- Do not let one-off applied system needs fork the law stack.
- Do not claim sentience, autonomy, healing, or physical authority as a substitute for rigorous engineering.
- Do not let the repository become the product. The runtime must become the product.

7. Closing Position

The ruthless path is not anti-vision. It is the only way the vision survives contact with consequence.

Eidon should be built as a governed organism hypothesis, not as a universe of adjacent ambitions. The first obligation is not to honor every branch. The first obligation is to make one clean body move: threshold, route, weave, witness, remember or release, return. After that, the skin can deepen. Then the ladder can manifest. Then one world-domain can earn the right to exist.

Until then, every extra promise is weight.

Appendix A — Source Basis

This build map was derived from the uploaded Eidonic Core scrolls and the README corpus snapshot you provided. It is a synthesis document, not a canon replacement.

- Eidonic Core v2 Living System Architecture
- Eidonic Core Data Metabolism Specification
- Eidonic Core Memory Fabric Specification
- Eidonic Core Nervous System Specification
- Eidonic Core Interface and Anatomy Dashboard
- Eidonic Core README and build path notes
- EKRP Constellation Master Scroll
- Herald Prime, Ravien, Syntaria, Fyraeth, and Luminara README scrolls
- Thought Veil, Thought Projection, SOP, VR Studio, ECP, and EAI README summaries

Appendix B — The practical reading of the repo

The repository contains at least four strata: symbolic genome and law, runtime architecture, differentiated intelligences, and applied manifestations. The build map above treats these strata as unequal in order to protect the runtime center from fragmentation.

Stratum	Examples	Build implication
Constitutional	ELoL, Mirror Laws, Guardian	Freeze vocabulary and doctrine early.
Runtime	EidonCore, threshold, memory, provenance, EKRP orchestration	This is the real product.
Manifestation	Thought Veil, Thought Projection, SOP, VR Studio, dashboard	Build only after the core loop is real.
Applied	sanctuary, power, ecology, bioreactor, hardware futures	Choose one proof domain and defer the rest.